

Your Project, Explained Simply

Everything you built, in plain language — no jargon, nothing assumed

This document explains your brain tumour project from the beginning. It assumes you know nothing about deep learning. Read it once and you will be able to explain every number in your report and defend every slide in your presentation.

How to use it: read sections 1 to 4 to understand what you built. Read section 5 the night before your review — it contains the exact sentences to say and the answers to the questions you will be asked.

1. What the project is, in one page

The starting point

You built a computer program that looks at a brain scan and draws an outline around the tumour. It works well — it gets about 82% agreement with what a doctor would draw.

Your mentor said: this is common. Thousands of students build exactly this. He was right.

The question you asked instead

A score of 82% describes an *average*. It tells you how the program does across hundreds of scans. It tells you nothing about the one scan a doctor is looking at right now.

Think of a weather app that is right 82% of the time. That sounds fine — until you realise it never tells you which days it is guessing. If it said "I am confident today" and "I have no idea today", you could plan around it. Without that, every forecast looks equally certain.

So you stopped trying to push 82% higher. You asked a different question: **when can this program be trusted, and can it tell you itself?**

What you did

You ran three experiments and trained 24 separate models to answer them.

- **Experiment 1** — does the program get worse at a hospital it has never seen before?
- **Experiment 2** — how much data does this actually need?
- **Experiment 3** — can the program tell when it is about to be wrong?

The answers were, in order: not for the reason everyone assumes; more than you have; and yes, mostly.

2. Words you will keep hearing

Dice score

A number from 0 to 1 that says how well two shapes overlap.

Imagine the doctor draws the tumour on tracing paper, and your program draws it on another sheet. Lay them on top of each other. If they match exactly, Dice = 1.0. If they do not touch at all, Dice = 0. Your program scores about 0.82, which means the shapes mostly line up but the edges differ.

Patient vs slice

An MRI is not one picture. It is a stack of flat pictures taken through the head, like slices of bread. One patient gives you 20 to 100 slices.

This matters because of **cheating**: if slice 11 of a patient is used for practice and slice 12 for the exam, the program has effectively seen the answer — those two slices look nearly identical. So you always split by *patient*, never by slice. Your code actually checks this and refuses to run if a patient ends up on both sides.

Train, validation, test

- **Train** — the practice questions the program learns from.
- **Validation** — questions you check progress on while studying.
- **Test** — the final exam. Never looked at until the very end.

Tumour-only Dice (important)

Many slices contain no tumour at all. On those, the program scores a perfect 1.0 just by drawing nothing — which is not a skill.

Counting those inflates your average by about 0.11. So you report two numbers: the usual one (0.93) and the honest one (0.82), and you use the honest one everywhere.

If asked why your score looks lower than other projects: because they are counting empty slices and you are not. Say that. It is a point in your favour, not against you.

Domain shift

The technical name for "the pictures look different at a different hospital, so the program gets confused." Different MRI machines produce slightly different-looking images, the way two cameras photograph the same room differently.

3. The three experiments

Experiment 1 — is the hospital the problem?

Your 110 patients came from 5 different hospitals. The hospital code is hidden in the folder name: TCGA_DU_5849_19950405. Almost nobody uses this.

Hospital	DU	HT	CS	FG	EZ	Total
Patients	45	34	16	14	1	110

The obvious test: hide one hospital completely during training, then test only on it. You did this for 4 hospitals. The score dropped every time — on average from 0.82 to 0.74.

Looks like proof, right? It is not.

The trap. Hiding hospital DU also removes 45 of your 110 patients. Hiding FG removes only 14. So you changed two things at once: which hospital is missing, *and* how much data is left. The drop could be from either.

What you did about it. For every hidden-hospital model you trained a twin with the exact same number of patients, but with the test patients picked randomly from all hospitals instead of just one. Same amount of data, only difference is whether a hospital is missing.

Like testing whether a new fertiliser works. If you move plants to a sunnier spot AND add fertiliser, you cannot tell which helped. So you keep a second pot in the same sunny spot with no fertiliser. That second pot is your control.

The result: the twins scored about the same. Site effect = **-0.0020** — basically zero. Three of the four hospitals actually scored *better* unseen.

The clearest single proof: the twin for DU excluded no hospital at all, and still scored 0.6915 — *lower* than the hidden-DU model at 0.7052.

Experiment 2 — how much data do you need?

You trained the same program on 15, 30, 45, 60 and 75 patients, three times each with different patients picked, always tested on the same fixed exam.

Training patients	15	30	45	60	75
Dice	0.668	0.726	0.750	0.786	0.798

It climbs fast at first, then flattens. The maths says it would top out near **0.918**, and you would need about **96 patients** to get close — more than this dataset contains.

The second thing this gave you — and it is the most important number in your project. Running the same size three times with different patients gave different scores, varying by about ± 0.057 . That is pure luck-of-the-draw noise. Any effect smaller than 0.057 cannot be measured on this dataset at all.

Why that sinks the whole hospital question

Your 8 hospital models all trained on 55 to 82 patients — a range where the curve only moves by 0.032. But their actual scores spanned 0.131.

You are trying to weigh a letter on a bathroom scale. The letter weighs 20 grams. The scale jumps 200 grams depending on how you stand on it. You cannot measure the letter — not because the letter is not there, but because your instrument is too crude.

That is your main finding, and it is a warning to the whole field:

If you test on fewer than about 50 patients per hospital, you cannot detect domain shift. The random luck of which patients you picked is four times bigger than the effect you are hunting. Anyone reporting a hospital effect from a study that size is probably reporting noise.

Experiment 3 — can it tell when it is wrong?

This is the part that gets you marks, because it produces a feature people can see.

How it works. You show the program the same scan 8 times — rotated and flipped different ways — then turn every answer back the right way up. All 8 should land on top of each other.

Like asking the same person a question 8 times in slightly different words. If they give the same answer every time, they know it. If the answers wander, they are guessing.

When the 8 answers agree, the program is confident. When they disagree, it is unsure. You call that disagreement the **instability score**.

Why this is honest here: the program was *trained* with rotations and flips. It has every reason to not care about them. So if it suddenly does care, something is genuinely wrong — you are not just confusing it with something unfamiliar.

Then you proved it actually works. You took the 20% of scans the program was least sure about, set them aside for a doctor, and measured the score on everything it kept:

% handed to a doctor	0%	10%	20%	30%	40%	50%
Dice on what it kept	0.808	0.882	0.901	0.918	0.926	0.934

0.81 to 0.90 by handing over just one slice in five. That means the program is giving away exactly the cases it was about to get wrong. It genuinely knows what it does not know.

And where it fails — say this yourself

Out of 224 slices with tumours, 22 came back completely empty. They split into two groups:

- **11 were caught.** Some of the 8 views did find the tumour, they just disagreed too much to agree on an outline. Your app flags these as "possible missed finding."
- **11 were not caught.** All 8 views unanimously found nothing, and all 8 were wrong. There is no disagreement to detect.

This second group is about 5% of tumour slices, and it cannot be fixed by this method. If everyone agrees and everyone is wrong, there is no signal. That is a limit of the approach, not a bug in your code — and you measured exactly how big it is.

4. What you actually built

File	What it does
train.py	Trains one model
make_splits.py	Creates the 5 hospital experiments
make_control_splits.py	Creates the 4 control twins
evaluate_domain_shift.py	Scores all 9 and splits site vs data effect
learning_curve.py	Trains the 15 models for the data curve
analyze_learning_curve.py	Fits the curve, finds the noise floor
uncertainty.py	The 8-view confidence experiment
app.py	The live demo
smoke_test.py	Checks everything works before a demo

24 models trained in total. 9 for the hospital study, 15 for the data curve.

The demo app

Upload a brain scan and it gives you: the tumour outline, a heat map showing where it is unsure, the size, which side of the brain, how many of the 8 views found something, and a verdict — CONFIDENT, REVIEW, HIGH RISK, or POSSIBLE MISSED FINDING.

The verdict thresholds are not made up. They come from the 60th and 85th percentile of instability measured on 279 real test slices.

5. The night before your review

Your one-sentence answer to "what did you do?"

"Everyone builds a tumour segmentation model and reports one accuracy number. We asked when that model can actually be trusted — and found that the standard way of testing it cannot answer the question, while the model itself can tell you which individual predictions to doubt."

The four numbers to memorise

Number	What it means
0.8176	Our honest baseline. Matches the 0.82 the dataset authors reported.
−0.0020	The hospital effect, once data amount is controlled for. Basically zero.
0.057	The noise floor. Nothing smaller can be measured here.
0.81 → 0.90	What happens when the least-confident 20% go to a doctor.

Questions you will be asked, and what to say

"Your accuracy is lower than other projects."

"We exclude empty slices. A model scores a perfect 1.0 on a slice with no tumour by drawing nothing, which inflates the average by about 0.11. Our 0.82 is the honest number and it matches the published figure for this dataset."

"Your main finding is negative. Did the project fail?"

"The negative result is the finding. We showed the standard test is under-powered by a factor of four — which means studies that report a hospital effect at this scale are probably reporting noise. And we have a positive result too: the model can flag its own failures and that lifts accuracy from 0.81 to 0.90."

"Why not a newer architecture than U-Net?"

"Architecture was not the question. We held it constant deliberately so the only thing changing between our 24 models was the experimental condition. Changing the architecture would have confounded every comparison we made."

"Why test-time augmentation instead of MC Dropout?"

"MC Dropout needs dropout layers active at inference, and this U-Net has none, so it could not run without retraining. Test-time augmentation also suits this model better — it was trained with flips and rotations, so disagreement under those is genuine instability rather than an unfamiliar input."

"Can it tell what type of tumour it is?"

"No, and not from this dataset. Every patient here has the same diagnosis — lower-grade glioma — so there is nothing to classify between. Claiming otherwise would be dishonest."

"What about the 5% it misses silently?"

"That is the structural limit of disagreement-based uncertainty. If all eight views agree and all eight are wrong, there is no signal left. We measured it rather than hid it. Fixing it would need an orthogonal uncertainty source such as an ensemble of independently trained models — that is our stated future work."

Three things to do, not say

- Raise your limitations **before** anyone asks. Raised first they read as rigour; found first they read as holes.
- When you demo, show a failure case on purpose — the slice where the model misses the tumour and the system flags it anyway. That is the strongest moment you have.
- Never say "it works". Say what it does, on what data, and where it stops working. That is the whole character of this project.